Reverse Mode Auto-differentiation in diffoDL

Some scalar tensors:

```
Tensor x({{2}}, true);
Tensor y({{3}}, true);
Tensor a({{1}}, false);
a.constant();

Tensor z = x + y;
Tensor L = a / z;

L.backward();
```

What's happening underneath?

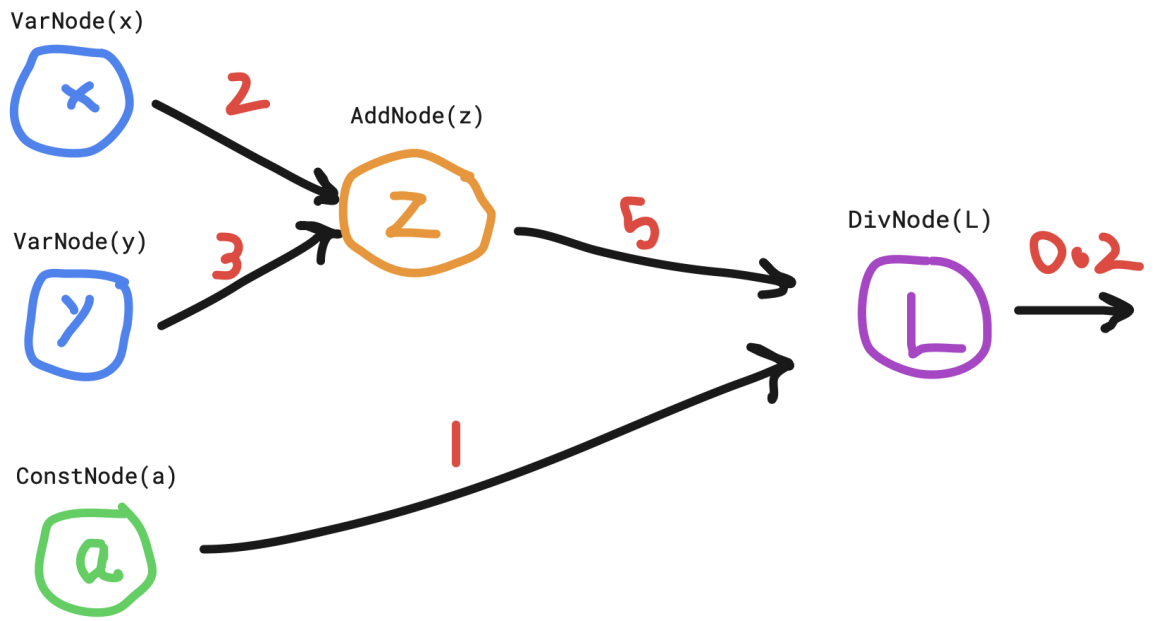
For requires_grad=true tensors, VarNodes are initialized.
For requires_grad=false tensors, ConstNodes are initialized (grad=0).

There exist Nodes corresponding to different operations as well.
These Nodes store metadata regarding variables, operations and cache required to compute gradients. They contain the actual backward_step() method that accumulates gradients to all inputs.

When z = x + y is computed, an AddNode is created.
When L = a / z is computed, a DivNode is created.

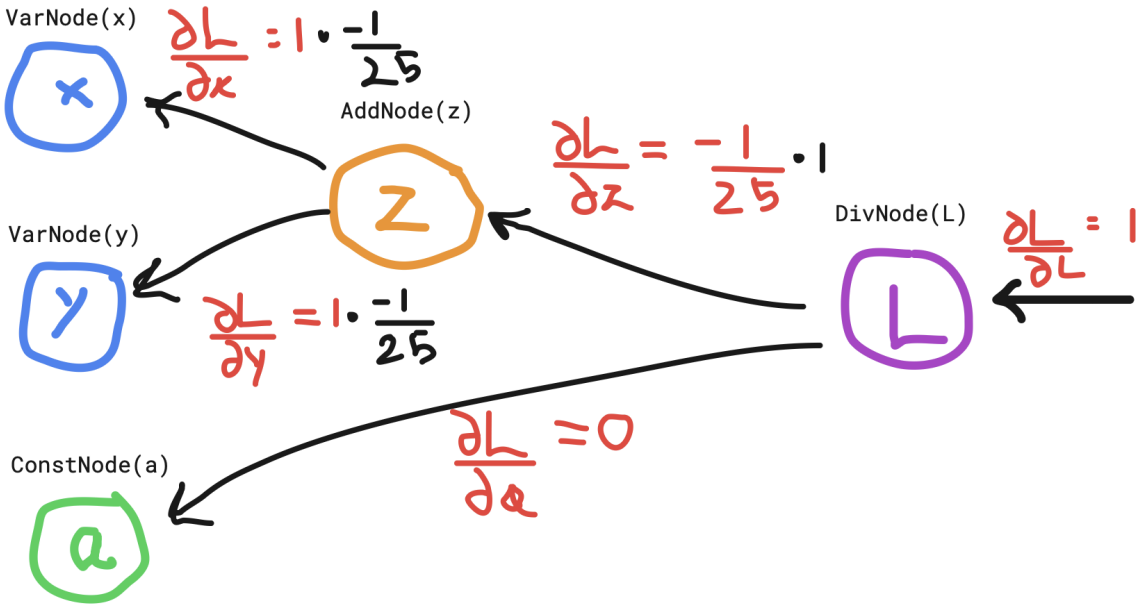
Every tensor stores their respective Node as an attribute called 'function'.
Eg. x and y store VarNodes, a stores a ConstNode, z stores an AddNode and L stores a DivNode

This is the forward pass that constructs the 'computation graph' i.e. links different nodes together with shared pointers



L.backward() triggers the backward pass where gradients are computed:

- L.backward() triggers the DivNode::backward_step()
- DivNode::backward_step() triggers AddNode::backward_step() and ConstNode::backward_step()
- ConstNode::backward_step() just sets gradient of a to 0
- AddNode::backward_step() triggers VarNode::backward_step() of x and y
- VarNode::backward_step() accumulates x.grad and y.grad
- When you call x.grad() and y.grad(), dL/dx and dL/dy are returned.



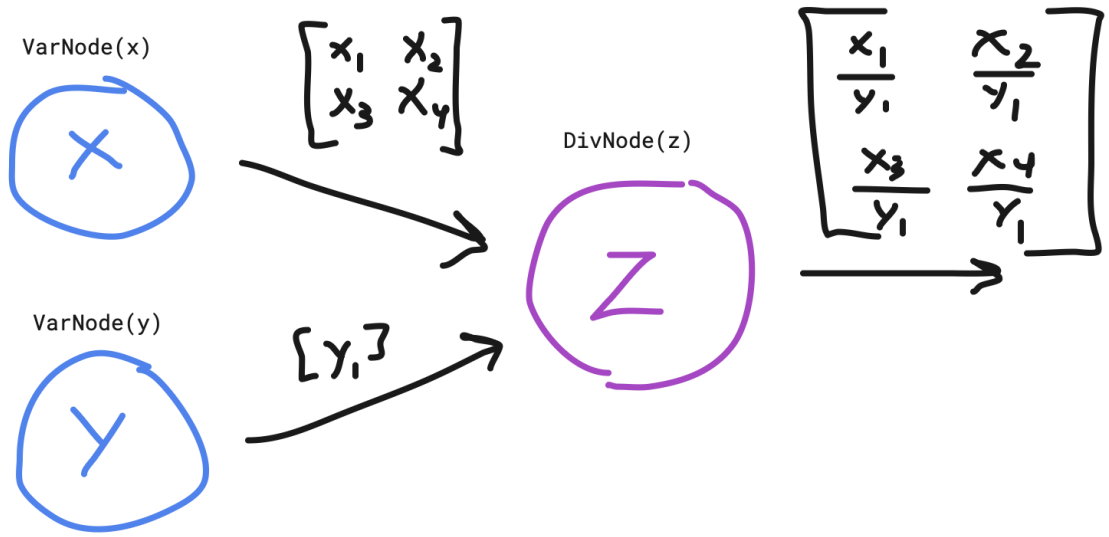
Vectorized version:

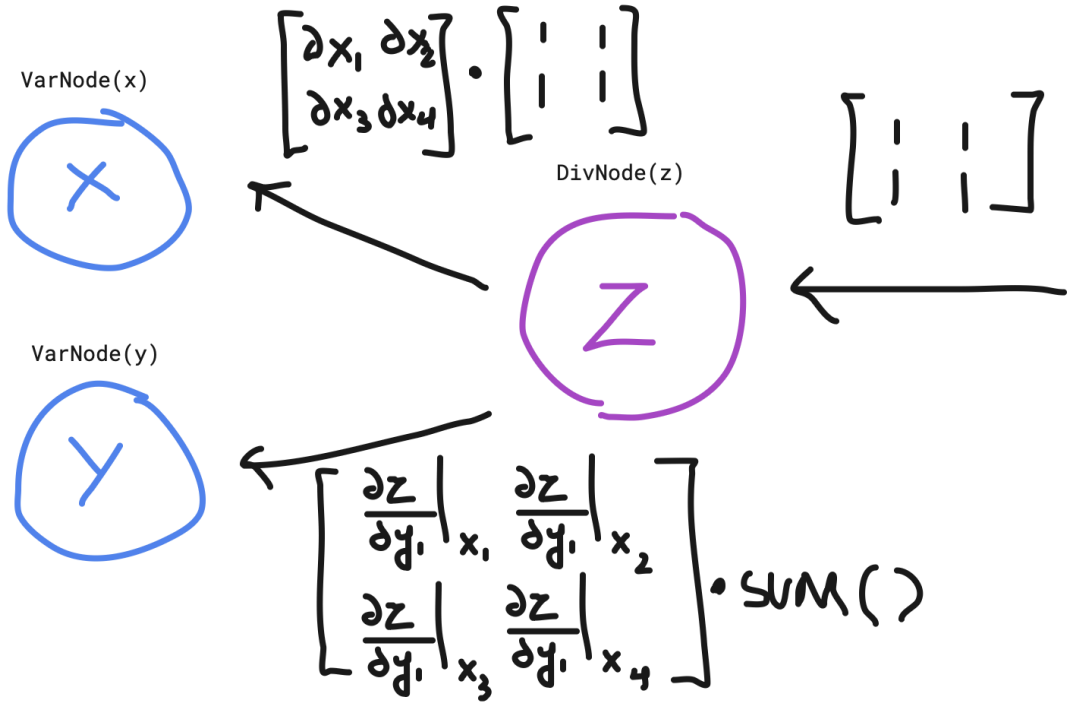
```
Tensor x({{1, 2}, {3, 4}}, true);
Tensor y({{2}}, true);

Tensor z = x / y;

z.backward();
```

When x / y is computed, y is first broadcasted to a 2x2 matrix and then element wise division takes place. Again VarNode(x) and VarNode(y) together help in forming DivNode(z)

When z.backward() is called. A seed of size 2x2 with just 1s is passed down to DivNode(z). DivNode(z) passes down dz/dx and dz/dy to VarNode(x) and VarNode(y). And then those accumulated to tensors x and y.
dz/dx and dz/dy are passed by reducing the dimension of grad to the dimension of x and y (de-broadcasting). For example in dz/dy, all elements of the 2x2 matrix of dz/dy are summed so as to form a 1x1 dz/dy matrix -> y.grad()

Forward Pass:



Reverse Pass:

$$\partial x_i = \frac{\partial z}{\partial x_i} = \frac{1}{y_1}$$